



→ list over here.
★ sorted(name) → Use to sort the things in particular order.

Issue :-

When we have list and we are taking input from user then as many times you run the program that many times you have to give input values. But what if I have huge code base than writing input values again and again is pure Headache. There comes in File I/O.

★ Open

File names.py :-

file = open("store.txt", "w")
↳ used to open or create a file
↳ Like dbl clicking app.
↳ read, write or append possible

file.write("Kavya")

↳ Once app open now write

file.close()

↳ To click save button .

'w' → overwrite everything written in file.

Tw
NAP
GIT FOR
MT
MP
Data
Sleep
BSS

Therefore, use append "a", if you add more names than now it don't overwrite it just append new ones.

```
name = input("What's your name?")
```

```
file = open("store.txt", "a")
```

```
file.write(f"{name} \n")
```

with:-

Many time people forgets to do file.close()
There where with arrives.

Syntax:-

```
with open("store.txt", "a") as file:
```

```
..... file.write(f"Kavya \n")
```

Automatically, closes as the last line will execute.

★ Read:-

```
with open("names.txt", "r") as file:
```

```
lines = file.readlines()
```

↙ returns list of all read values

```
for line in lines:
```

```
    print("hello, " line.rstrip())
```

↗ to remove all gaps or white space.

We can also do;

```
with open("names.txt", "r") as file:
```

reads
each
line
in
file.

↙ for line in file:

```
    print("hello, " , line.rstrip())
```

To sort the names inside file :

we can do:

↗ if not written anything than "r" by default.

```
with open("store.txt") as file:
```

```
    for line in sorted(file):
```

```
        print(line.rstrip())
```

or.

```
with open("store.txt") as file:
```

```
    lines = file.readlines()
```

```
for line in sorted(lines):  
    print(line.rstrip())
```

sorted("names", reverse=False)^{→ asc}
^{True → desc.}

With a csv file:-

with open("students.csv") as file:

..... for line in file:

..... ^{name, house}
..... ~~row~~ = line.rstrip().split(",")

..... print(f"row {row} is in row {row}")
or

print(f"{name} is in {house}")

To sort by name:-

```
students = []  
  
with open("students.csv") as file:  
    for line in file:  
        name, house = line.rstrip().split(",")  
        student = {}  
        student["name"] = name  
        student["house"] = house  
        students.append(student)  
  
for student in students:  
    print(f"{student['name']} is in {student['house']}")
```

- Unfortunately, we cannot sort the students as we had prior because each student is now a dictionary inside of a list. It would be helpful if Python could sort the `students` list of `student` dictionaries that sorts this list of dictionaries by the student's name.
- To implement this in our code, make the following changes:

```
students = []

with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",")
        students.append({"name": name, "house": house})

def get_name(student):
    return student["name"]

for student in sorted(students, key=get_name):
    print(f"{student['name']} is in {student['house']}")
```

→ got automatically
call in each of the
dictionary by
sorted

Notice that `sorted` needs to know how to get the key of each student. Python allows for a parameter called `key` where we can define on what "key" the list of students will be sorted. Therefore, the `get_name` function simply returns the key of `student["name"]`. Running this program, you will now see that the list is now sorted by name.

Key = `lambda student : student["name"]`

↓
anonymous function.
with
no name.

CSV

`import csv`

`students = []`

with open (" student . csv ") as file:

`reader = csv . reader (file)`

for `name, home`
`row` in reader:

`student . append ({"name": row[0],`
`"home": row[1]})`

★ PIL ↪ To store image file.
↳ pillow.readthedocs.io

```
import sys
```

```
from PIL import Image
```

```
images = []
```

```
for arg in sys.argv[1:]:
```

```
    image = Image.open(arg)
```

```
    images.append(image)
```

```
images[0].save(
```

```
    "costume.gif", save_all=True, append_images=[images[0]]
```

```
    duration=200, loops=6
```

```
)
```

